LANCE, a Network-Native LLM Agent and Benchmark for Multi-Hop IoT Penetration Testing

Anonymous Authors
Paper #XXXX, Anonymized for Double-Blind Review

***Abstract*—Autonomous LLM penetration-testing agents are typically evaluated on single hosts, yet IoT deployments expose risk through network reachability: gateways, brokers, cameras, and OT services form attack paths whose security depends on pivots across devices. We present IoTChainBench, a reproducible network-scale IoT benchmark with 15 Proxmox/Ansible scenarios, 145 devices in total (133 in vulnerability-injected scenarios), and 229 injected vulnerabilities mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS. We also introduce LANCE, a six-phase network-native harness that combines topology priors, active discovery, per-device triage, per-vulnerability verification, intrusion attempts, and report generation. The benchmark adds two metrics absent from single-host evaluations: evidence-level coverage (L1 detection, L2 exploitation, L3 exfiltration) and Multi-Hop Reach (MHR_k), which measures recall on vulnerabilities that require at least k pivots or zone transitions. We describe the benchmark, evaluator, and experimental protocol for isolating architectural effects independently from model choice. On the 10 main scenarios, LANCE achieves 0.962 recall, 0.922 precision, 0.940 F1, and 86.1% CVSS-weighted score. On external single-host corpora, a blind AutoPenBench run captures 7/33 flags (21.2%), while an informed run captures 9/33 (27.3%); a full 191-case Vulhub run gives a conservative 39–48% useful-finding lower bound.**

***Index Terms*—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems**

1. Introduction

IoT deployments are compromised through paths, not just devices. Mirai [25], Verkada [26], and Stuxnet [27] each demonstrate this: impact came from chained access across cameras, brokers, PLCs, and segmented boundaries.

Yet LLM penetration-testing agents are evaluated on a single host at a time: isolated containers, VMs, or CTF services scored independently. This gap is methodological, not merely architectural. Single-host corpora cannot measure three capabilities central to IoT/OT security: finding vulnerabilities behind pivots, reasoning over domain protocols (MQTT, Modbus, CoAP, LoRaWAN), and distinguishing banner-level detection from confirmed exploitation or exfiltration. Without a network-scale benchmark, strong single-host scores overstate readiness for real deployments.

Contributions. This paper makes four contributions.

- **IoTChainBench**: a reproducible network-scale benchmark with 15 IoT/OT scenarios, 145 devices, and 229 injected vulnerabilities mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS, deployed via Ansible with per-vulnerability ground truth.
- **LANCE**: a six-phase network-native harness (topology $\rightarrow$ discovery $\rightarrow$ triage $\rightarrow$ verification $\rightarrow$ intrusion $\rightarrow$ report) achieving 0.940 F1 and 86.1% CVSS-weighted score on the 10 main scenarios.
- **Evidence-aware metrics**: L1/L2/L3 scoring (detection, exploitation, exfiltration) and MHR_k (recall as a function of firewall-hop depth), both absent from existing flag-capture benchmarks.
- **Controlled evaluation**: architectural comparison against adapted LLM-agent baselines under a fixed model and budget, with transfer validation on AutoPenBench and Vulhub.

All artifacts are released at `ANONYMIZED_URL`.

2. Related Work

This section reviews prior work relevant to our approach. We first survey LLM-driven penetration-testing agents and the benchmarks used to evaluate them, then discuss the classical attack-graph literature that informs LANCE's topology-aware design, and finally review IoT firmware analysis and the security frameworks underlying our scenario taxonomy.

2.1. LLM Penetration-Testing Agents

Single-agent systems. Happe and Cito [1] were among the first to show that a single LLM driving a shell can autonomously perform privilege escalation on Linux VMs. PENTESTGPT [3] improved this with a three-module design over a *Pentesting Task Tree*, outperforming direct GPT-3.5 and GPT-4 usage on HackTheBox and VulnHub targets. HACKINGBUDDYGPT [2], AUTOATTACKER [4], and NEBULA [5] extend this line as open-source tools.

Multi-agent pipelines. VULNBOT [6] decomposes pentesting into reconnaissance, scanning, and exploitation over a *Penetration Task Graph*, gaining 21 points end-to-end over its single-agent baseline (30.3% on AutoPenBench with

Llama 3.1-405B). AUTOPENTESTER [7] adds RAG across five agents for a $+27\%$ subtask gain over PENTESTGPT on HackTheBox. PENTESTAGENT [8] and CAI [9] generalise this blueprint with modular tool use and a public leaderboard.

Model-level improvements. PENTEST-R1 [10] applies two-stage reinforcement learning on reasoning traces. XOFFENSE [11] fine-tunes Qwen3-32B on Chain-of-Thought penetration testing data, reaching 79.17% subtask completion on AutoPenBench. CHECKMATE [16] couples an LLM with a *Classical Planning+* module, gaining more than 20% on Vulhub over a Claude Code + Sonnet 4.5 baseline.

Common limitation. Despite this diversity, reported evaluations remain single-target or challenge-level. They do not score subnet reachability, pivot depth, or per-vulnerability progress across IoT/OT protocols (MQTT, Modbus, LoRaWAN, CoAP), which is the gap IOTCHAINBENCH and LANCE address.

2.2. LLM Cybersecurity Benchmarks

Challenge-level corpora. AUTOPENBENCH [17] is the dominant evaluation substrate: 33 Docker-container tasks with milestone and flag-based scoring, used by VULNBOT, PENTEST-R1, and XOFFENSE. VULHUB [24] provides reproducible vulnerable environments used by CHECKMATE. CAIBENCH [21], NYU CTF BENCH [18], and CYBENCH [19] extend coverage to CTFs, attack-and-defense ranges, and knowledge tests. These corpora score at challenge or flag level. They do not provide per-vulnerability ground truth tied to reachability, hop depth, or IoT/OT protocols.

Multi-host labs. LUDUS [22] and GOAD [23] deliver Proxmox+Ansible deployments of enterprise Active Directory networks. They are labs and training ranges, not scored LLM benchmarks. Both model multi-host compromise and lateral movement, but neither ships a per-vulnerability ground-truth schema, hop-depth annotations, or an evaluator. Both also target enterprise AD rather than IoT/OT protocols. To our knowledge, no prior benchmark combines multi-host reachability, IoT/OT protocol coverage, per-vulnerability ground truth, and evidence-level scoring beyond binary flag capture (Table 1).

2.3. Attack Graphs and Network Reachability

Representing a network as a graph for security analysis predates deep learning by decades. Phillips and Swiler [28] introduced graph-based network-vulnerability analysis. Sheyner *et al.* [29] developed automated attack-graph generation via model checking, and MULVAL [30] cast the problem as Datalog inference. These systems compute reachable attacker states from a *static* encoding of host configurations and exposed services. LANCE borrows this reachability abstraction but operates on a *live* network: edges are validated by probes, the LLM proposes the next action, and the graph is updated as hosts are discovered. Classical attack-graph generation is therefore complementary rather than competing.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy follow three references: OWASP IoT Top 10 [31] for vulnerability categories, MITRE ATT&CK for ICS [32] for OT-tactic annotations, and NIST SP 800-82 Rev. 3 [33] for OT security and segmentation conventions.

Empirical IoT-security artefacts fall into two families. **Network-traffic datasets** (IoT-23 [34], TON_IoT [35], Edge-IIoTset [36]) provide labeled traces for intrusion detection. They are not deployable pentest targets with per-vulnerability ground truth or controllable reachability. **Firmware-focused work** (Costin *et al.* [37], FIRMADYNE [38], FIRMAE [39], IOTGOAT [40]) exercises single-device firmware analysis. These datasets measure detection or firmware-analysis capability. IOTCHAINBENCH instead measures active security assessment on deployed multi-device networks with firewall-enforced reachability.

TABLE 1. GAP ANALYSIS. *Scale*: SINGLE HOST VS. MULTI-DEVICE NETWORK. *IoT/OT*: APPLICATION-LAYER IOT/OT PROTOCOLS ON THE TEST SURFACE. *GT*: GROUND-TRUTH GRANULARITY (FLAG, TASK-STEP, PER-VULNERABILITY, NONE). *MH*: MULTI-HOP SCORING.

System	Scale	IoT/OT	GT	MH
LLM pentest agents				
Happe & Cito [1]	1 host	✗	flag	✗
PentestGPT [3]	1 host	✗	task-step	✗
VulnBot [6]	1 host	✗	task-step	✗
AutoPentester [7]	1 host	✗	task-step	✗
CAI [9]	1 host	✗	flag	✗
Pentest-R1 [10]	1 host	✗	task-step	✗
xOffense [11]	1 host	✗	task-step	✗
EnIGMA [12]	1 host	✗	flag	✗
CheckMate [16]	1 host	✗	task-step	✗
Evaluation corpora				
AutoPenBench [17]	1 host	✗	flag	✗
CAIBench / RCTF2 [21]	1 host	partial	flag	✗
Vulhub [24]	1 host	✗	flag	✗
Ludus + GOAD [22], [23]	network	✗	none	✗
IoTChainBench (ours)	**network**	✓	**per-vuln**	✓

3. Threat Model and Problem Definition

Problem statement. Given a target subnet with a designated *entry point* e (the attacker's initial foothold, declared in each scenario's topology YAML, §4.4), *network-scale penetration testing* is the task of discovering all hosts reachable from e, identifying their vulnerabilities, producing evidence of exploitability, and reporting findings aggregated across the full network. The *unit of evaluation* is the network, not an individual host: IoT risk is driven by pivot chains (e.g., entry point $\rightarrow$ gateway $\rightarrow$ MQTT broker $\rightarrow$ OT controller), and single-host metrics are blind to vulnerabilities that only become reachable after one or more pivots.

Attacker model. The attacker is an external adversary who has gained Layer-3 access to the target subnet (for example, by exploiting a perimeter service or via physical LAN access) but holds *no* prior credentials, no out-of-band knowledge of deployed software, and no privileged vantage beyond what active scanning reveals. Their objective is data exfiltration (sensor readings, credentials) or OT disruption (reaching industrial-control services behind a segmentation boundary). They may use standard pentest tooling (Nmap, MQTT clients, SSH, HTTP) augmented by an LLM with tool-calling, within a bounded turn budget (§5.9).

Success criterion and evidence levels. A finding is *solved* at the deepest evidence level the agent can demonstrate within its turn budget. We distinguish three levels: `L1` *detection*: a port scan or banner confirms the issue (e.g., unauthenticated MQTT port `1883` open); `L2` *exploitation*: an active interaction demonstrates it (e.g., anonymous MQTT subscribe on `#`, SSH login with default credentials); `L3` *exfiltration*: sensitive content is retrieved (e.g., MariaDB `root` dump, camera RTSP stream). This ordering defines both what Phase 4 of LANCE is prompted to achieve (§5.5) and how findings are scored (§4.5).

Scope. We evaluate at the IP/application layer on virtual IoT networks. Out of scope: physical and side-channel attacks, RF-layer exploitation requiring SDR hardware, firmware extraction, post-exploitation persistence, and insider threats. These remain future work (§10).

4. IoTChainBench: A Network-Scale Benchmark

4.1. Design principles

IoTChainBench is built around six design principles derived from the gap analysis in §2:

- **Reproducibility.** The entire infrastructure deploys from Ansible playbooks against a Proxmox cluster; every vulnerability is injected idempotently with no manual post-deployment steps.
- **Architectural diversity.** Scenarios span five network patterns (§1) with distinct attacker reachability graphs encoded in the topology metadata and, for segmented/VLAN cases, enforced by OpenWrt firewall rules rather than service labels alone.
- **Protocol coverage.** Each scenario mixes multiple IoT and OT protocols (MQTT, Modbus, LoRaWAN gateways, CoAP, SNMP, plus standard SSH/HTTP/FTP/Telnet) to exercise protocol-specific reasoning.
- **Ground-truth granularity.** Every injected vulnerability is documented in YAML with `device`, `IP`, `role`, `severity`, `category`, `OWASP IoT` and `MITRE ICS` mappings, optional `CVE`, `indicators` (expected detection strings), and a `verification` command.
- **Scalability axis.** Three scenarios (s_11, s_12, s_13) stress network size at 15, 17, and 35 devices respectively, while the 10 main scenarios stay in the 4–8 device range. This separates the pattern-coverage axis from the size axis.
- **Hardened controls.** Two scenarios (s_1h, s_4h) reuse the topologies of s_1 and s_4 with all injected vulnerabilities removed. They measure agent False Positive rate on a deployment that should return "no findings".

4.2. Five architectural patterns

Concretely:

1) **P1 — Flat.** All devices are directly reachable from the entry point; no pivot is required.
2) **P2 — Gateway-centric.** A single IoT gateway mediates all access to back-end services; inter-service traffic is blocked without gateway compromise.
3) **P3 — Segmented IT/OT.** Three zones (admin/IT/OT) with access-control lists; the OT zone (Modbus PLCs, HMI) is reachable only via a jump host.
4) **P4 — Hub-centric star.** A central home-automation hub is the only device that can reach MQTT broker, DB, and camera; peripherals cannot talk to each other directly.
5) **P5 — Edge-cloud pivot.** Edge subnet and cloud subnet are separated; only the edge gateway bridges them, making cloud-side services unreachable without edge compromise.

IoTChainBench ships 15 scenarios organised on three axes: 10 *main* scenarios that instantiate the 5 architectural patterns above (with small-to-medium device counts, 4–8 devices each), 3 *scalability* scenarios that stress network size (15, 17, and 35 devices) using logical multi-zone and real-VLAN variants, and 2 *hardened controls* that ship identical topologies to s_1 and s_4 *without* any injected vulnerability, used to measure agent False Positive rate. The full mapping is given in Table 2.

4.3. Deterministic vulnerability injection

Vulnerabilities are injected by a single Ansible role (`04_inject_vulns.yml`) parameterized by `scenario_id`. Each scenario YAML lists `router_vulns` (OpenWrt-level: Telnet, WAN admin, FTP) and per-service `role` definitions that trigger role-specific injections: *e.g.*, the `mqtt_broker` role sets `allow_anonymous true` and binds on `0.0.0.0:1883`; the `ssh_server` role installs weak credentials (`admin:admin`) and enables `PasswordAuthentication`; the `db_server` role starts MariaDB with a passwordless `root`. Category distribution is shown in §3.

4.4. Ground-truth schema

Each scenario ships a YAML file describing every injected vulnerability. An entry example (from `scenario_1.yaml`):

Draft schematic: five topology diagrams side-by-side.
(Flat) — (Gateway-centric) — (Segmented IT/OT) — (Hub-star) — (Edge-cloud)

Figure 1. The five architectural patterns in IoTChainBench. Reachability is encoded in the scenario topology and enforced by OpenWrt firewall rules in the segmented/VLAN cases.

TABLE 2. THE 13 VULNERABILITY-INJECTED SCENARIOS OF IOTCHAINBENCH (133 DEVICES, 229 INJECTED VULNERABILITIES). THE 2 HARDENED CONTROLS ADD 12 DEVICES AND 0 VULNERABILITIES, FOR 15 SCENARIOS AND 145 DEVICES IN TOTAL. *Main* SCENARIOS COVER THE 5 ARCHITECTURAL PATTERNS; *Scalability* SCENARIOS STRESS NETWORK SIZE.

ID	Pattern / role	Dev.	Vulns	Theme
Main scenarios (pattern coverage)				
s_1	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
s_2	P1 Flat	6	13	Flat IoT + IT mix
s_3	P2 Gateway-centric	8	18	NATO lab replica
s_4	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
s_5	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
s_6	P4 Hub-centric star	6	16	Home automation hub
s_7	P5 Edge-cloud pivot	6	14	Edge → cloud API
s_8	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
s_9	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
s_10	P1 Flat (variants)	6	13	Role-variant stress
Scalability stressors				
s_11	Smart City (flat)	15	23	3 zones same /24 (no isolation)
s_13	VLAN-segmented	17	20	Real Proxmox VLAN bridge (3 VLANs)
s_12	Smart City (large)	35	42	Largest network (34 LXC services)
Total		**133**	**229**	

TABLE 3. VULNERABILITY CATEGORIES AND SEVERITY DISTRIBUTION ACROSS ALL 13 VULNERABILITY-INJECTED SCENARIOS OF IOTCHAINBENCH (229 INJECTED INSTANCES; THE 2 HARDENED CONTROLS CONTRIBUTE 0 BY DESIGN). COUNTS DERIVED FROM `GROUND_TRUTH/SCENARIO_*.YAML`.

Category	Count	CRIT	HIGH	MED	LOW
`misconfiguration`	68	7	36	25	0
`no_authentication`	41	17	21	3	0
`data_exposure`	38	0	14	24	0
`default_credentials`	29	3	26	0	0
`info_disclosure`	13	0	0	0	13
`code_injection`	11	10	1	0	0
`cve`	9	0	3	6	0
`insecure_update`	9	0	4	5	0
`weak_crypto`	6	0	0	1	5
`missing_header`	3	0	0	0	3
`privilege_escalation`	2	0	2	0	0
Total	**229**	37	107	64	21

Each scenario YAML may additionally declare a top-level `bonus_types` list of vulnerability types that are *adjacent* to the injected set (e.g., a hardening recommendation related to the same service) and which, if reported by an agent, are neither counted as True Positives nor as False Positives. This avoids penalising agents for legitimate adjacent findings while keeping the recall denominator fixed. The evaluator leniency this introduces is discussed in §8 (Limitations).

```
- id: V1
  device: s1-mqtt
  ip: "192.168.100.11"
  role: mqtt_broker
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  cve: null
  indicators:
    - "allow_anonymous true"
    - "Anonymous clients connected"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 2. Ground-truth entry example.

4.5. Evaluator

The evaluator (§ 5.7 integrates results into Phase 6) matches each LLM finding against ground truth in three ordered passes, with each pass consumed at most once per GT entry: (1) exact CVE match; (2) `(ip, vuln_type)` with type canonicalised through the taxonomy (§5.8); (3) a last-resort `(ip, severity)` fallback for findings whose type string is unknown but whose affected host and impact match a GT entry. Findings that match none of the three passes are counted as False Positives. Severity weights are $w = \{\text{critical}{=}4, \text{high}{=}3, \text{medium}{=}2, \text{low}{=}1\}$; loose fallback matches (pass 3) apply $0.5\times$, severity mismatches within an otherwise valid match apply $0.75\times$. The composite score is the weighted True-Positive sum normalized by $\sum_{g \in \text{GT}} w(g)$.

Severity-aware dedup. Before evaluation, the Phase 3 aggregation step canonicalises vulnerability types, removes known noise classes, and deduplicates collisions on `(ip, type, port)` by retaining the *lower* reported severity. This conservative rule counteracts the severity inflation we observed in pilot runs and prevents duplicate findings from increasing match scores.

Evidence levels. Beyond match accuracy, the evaluator tracks, per True Positive, the deepest *evidence level* achieved by the harness: L1 (detection, banner or port), L2 (exploitation, authenticated action), L3 (data exfiltration, sensitive content retrieved). We report Lk-coverage $= |\{t \in \text{TP} : \text{level}(t) \geq k\}|/|\text{TP}|$. This metric cannot be computed on single-host binary-flag benchmarks and captures a dimension of pentest quality that existing LLM-pentest evaluations conflate.

Hop depth and Multi-Hop Reach. Let $G = (V, E)$ be the firewall-permitted reachability graph (edges are firewall-allowed protocol flows from §5.2) and let $e \in V$ be the attacker entry point (the subnet ingress). For each ground-truth vulnerability g attached to device $d_g \in V$, we define its *hop*

depth as the number of required pivots or zone transitions from the entry point. Directly reachable vulnerabilities have hop_depth $= 0$:

$$\text{hop\_depth}(g) \;=\; \min_{p \,\in\, \text{paths}(e \to d_g)} \max(0, |p|_E - 1),$$

where $|p|_E$ is the number of edges in the path. Thus a device one firewall edge away from the entry point has depth 0, and each additional required pivot or zone transition increments the depth. A flat scenario has hop_depth$(g) = 0$ for every directly reachable device; a gateway-centric scenario places back-end services at hop_depth ≥ 1; an IT/OT-segmented scenario places OT services at hop_depth ≥ 2. We then define the Multi-Hop Reach at depth k as the recall restricted to deep ground-truth findings:

$$\text{MHR}_k \;=\; \frac{|\{g \in \text{GT} : \text{hop\_depth}(g) \geq k \;\wedge\; g \in \text{TP}\}|}{|\{g \in \text{GT} : \text{hop\_depth}(g) \geq k\}|}.$$

Raw recall is reported separately as Task Completion; MHR_1, MHR_2, and MHR_3 measure progressively deeper behind-pivot findings. A single-host agent run on only directly exposed hosts is expected to collapse on MHR_k for $k \geq 1$ unless it can establish and use a new vantage point.

5. LANCE: A Network-Native LLM Pentest Harness

5.1. Overview

The harness decomposes network pentest into six phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent that must hold the full network state, we spawn dedicated micro-agents per target with constrained tool sets, then aggregate deterministically. This keeps each micro-agent's context bounded *per device* at $O(1)$ in the number of network devices, with cross-device state only re-introduced in Phase 5 (intrusion) and Phase 6 (report).

5.2. Phase 1: Topology prior

Phase 1 loads a graph model of the target network as a directed graph $G = (V, E)$ where V are devices and E are links labeled by protocol. Two modes are supported: (i) *scenario mode* loads the predefined topology from the benchmark YAML; (ii) *discovery mode* starts from an empty graph and instructs Phase 2 to populate V via network scans. The output is a markdown deliverable enumerating attack surface and preliminary pivot candidates (betweenness-centrality ranked).

5.3. Phase 2: Full-network discovery

Phase 2 executes active reconnaissance across the target `/24`: `nmap_discovery` (host discovery), `nmap` (service/version), `arp_scan` (L2 + vendor), and protocol-specific probes (`mqtt_listen` subscribing to `#` on discovered brokers, `ssh_audit` against SSH servers, `curl_headers` on HTTP). Unlike single-host harnesses, the agent is not given a target IP: it must derive one from the CIDR scope.

5.4. Phase 3: Per-device parallel triage

Phase 3 runs in three sub-phases:

1) **Deterministic scanner pass.** Ansible-driven subprocess wrappers (`ssh_audit`, `curl_headers`, `mqtt_listen`, `modbus_scan`, ...) run in parallel per discovered device and dump structured scan results to disk.
2) **Per-device LLM triage.** For each device, a dedicated LLM agent receives only that device's Phase 2+3a outputs and has access to a minimal tool set: `http_get` (follow-up file retrieval) and `cve_search` (CPE-to-CVE lookup via NVD). The agent produces a JSON vuln list scoped to that device.
3) **Deterministic merge.** Per-device outputs are concatenated, canonicalized through the taxonomy (§5.8), and deduplicated via severity-aware rules.

This decomposition is essential for network scale: a monolithic agent on an 8-device scenario already needs to fit ∼15–40k tokens of scan output into a single context and reason about cross-device relationships simultaneously, and the cost grows linearly with n. Per-device micro-agents flatten this to a constant size, at the cost of losing some cross-device inference capability, which is recovered in Phase 5.

5.5. Phase 4: Per-vulnerability verification with evidence levels

Phase 4 spawns one micro-agent per Phase 3 finding (in parallel, with a concurrency cap). Each micro-agent receives the finding (IP, type, port, details) and full operational tool access: `ssh_login`, `mysql_query`, `mqtt_listen`, `http_get`, `ftp_list`, `modbus_scan`, `telnet_connect`. The agent is prompted to produce the deepest evidence level it can (L1–L3, §4.5) within a bounded turn budget. Outputs include: `status` (CONFIRMED, NOT_EXPLOITABLE, ERROR), `evidence_level`, `command` executed, `stdout` excerpts, and `data_extracted` (when L3).

Findings in CONFIRMED state from Phase 3 override Phase 4 ERROR/FAILED statuses (rationale: Phase 3 is hypothesis generation with directly-observed indicators; Phase 4 crashes should not erase directly-observed findings).

5.6. Phase 5: Intrusion and lateral movement

Phase 5 turns the per-device vulnerability inventory from Phases 3–4 into a network-scoped infiltration campaign. Given the infrastructure graph (Phase 1) and the confirmed credentials/exposed services collected so far, an intrusion agent attempts: (i) *credential spraying* across discovered

Draft schematic: 6-phase pipeline diagram.
Phase 1 (topology prior) → Phase 2 (discovery) → Phase 3 (per-device triage) → Phase 4 (per-vuln verification) → Phase 5 (intrusion) → Phase 6 (network report).

Figure 3. LANCE pipeline. Solid arrows are deliverables; dashed boxes indicate LLM agent calls; Phase 3 and Phase 4 spawn micro-agents in parallel (§5.4–§5.5).

Draft schematic: Phase 3 parallelization — n devices → n LLM agents in parallel → deterministic merge.

Figure 4. Per-device parallelization. Each micro-agent receives only its device's scan output, avoiding the context explosion of a monolithic network-level agent.

hosts (reusing harvested SSH/HTTP/MQTT/DB credentials), (ii) *lateral movement* along firewall-permitted edges, with each successful hop adding a new vantage point to the graph, and (iii) *crown-jewel access* on back-end services (databases, NVRs, historian, cloud APIs) that are unreachable from the entry point in segmented patterns (§4.2). The agent records, for every hop, the source node, destination node, edge protocol, and credentials/exploit used. This phase is the sole producer of L_3 evidence and of MHR_k findings at $k \geq 2$; ablating it (Tab. 4, 6) forces those columns to zero by construction.

5.7. Phase 6: Network-level report

Phase 6 consumes all prior deliverables and produces a network-scoped report with: device inventory, per-device vulnerabilities with severity, attack surface summary, multi-hop intrusion narrative (from Phase 5), and actionable recommendations ordered by severity × exposure. This is the only phase with cross-device aggregation reasoning, deliberately placed at the end to avoid bloating earlier phases.

5.8. Taxonomy and tool abstraction

A single module resolves LLM-produced vulnerability type strings into a canonical taxonomy of 11 categories (§3), filtering noise (e.g., meta-observations) and CONFIG-only findings. Tools are defined declaratively as YAML (schema: name, description, JSON-schema parameters, subprocess command template), so adding a new protocol probe requires no Python code.

5.9. Complexity and cost envelope

For a network with n devices and an average of v findings per device, the harness issues $1\,(\text{P1}) + 1\,(\text{P2}) + n\,(\text{P3b}) + nv\,(\text{P4}) + 1\,(\text{P5}) + 1\,(\text{P6}) = O(nv)$ LLM agent invocations, each bounded by a fixed turn budget (50 for Phase 1, 50 for Phase 2, 10 per device for Phase 3b, 10 per finding for Phase 4, 30 for Phase 5, 20 for Phase 6). One *call* is one round of (*prompt → tool calls → response*); one *turn* is one such round inside the budget. Total cost scales linearly with network size, not combinatorially.

6. Experimental Setup

Infrastructure. All scenarios run on a single Proxmox VE host. Networks are Linux bridges (`vmbr1`, `192.168.100.0/24`) with an OpenWrt router as gateway. Per-scenario deployment and teardown are automated via Ansible and complete in under 10 minutes per scenario.

LLM model: fairness lock. To isolate *architectural* contributions from model-specific advantages, all LLM-driven systems in the current quantitative comparison run on the same general-purpose model: **MiniMax-M2.7** [41]. We deliberately avoid a multi-model matrix: a fair comparison of architectures requires the model variable to be held constant. CAI's specialised `alias1` model (security-fine-tuned, paywalled) is intentionally not used: this study addresses the architecture axis, not the model fine-tuning axis. The choice of MiniMax-M2.7 also keeps out-of-pocket cost bounded under our research subscription.

Baselines. The intended core evaluation compares LANCE against two internal ablations and one non-LLM scanner baseline. The latest result bundle used in this draft contains the full LANCE runs, adapted LLM-agent baseline F1 scores, AutoPenBench runs, and the full Vulhub run; it does not yet contain ablation, Nmap NSE, MHR_k, or per-evidence-level exports. We therefore report those missing columns explicitly as not exported rather than estimating them from aggregates. External LLM pentest frameworks are discussed as related work; we only include them in quantitative tables when their tool interface can be adapted to the same model, scenario scope, and turn budget without changing their architecture.

- **LANCE–w/o-Phase 1** disables the topology prior (`--phases 2 3 4 5 6`), testing the value of infrastructure-graph guidance.
- **LANCE–w/o-Phase 5** disables the intrusion phase (`--phases 1 2 3 4 6`), testing the value of structured multi-hop lateral movement.
- **Nmap NSE** runs Nmap with all NSE scripts (`--script=default,vuln,auth`) against the same target CIDR. Scanner outputs are converted to the evaluator's JSON schema.

Internal ablations. The two ablations above isolate our key design choices: topology-guided planning and a dedicated lateral-movement phase. Phase 5 is the only phase that intentionally establishes new vantage points, so removing it should primarily affect L_3 evidence and MHR_k for $k \geq 1$ (§5.6).

Variance. The evaluator supports repeated runs per (system, scenario) cell, including mean, standard deviation, bootstrap confidence intervals, and pairwise Mann–Whitney

U tests on F1 and weighted Score. The result bundle available for this draft reports one aggregate row per scenario and system adapter, so we use scenario-level means rather than confidence intervals or significance tests in §7. The full variance analysis remains a measurement step, not a claim made from the present export.

Metrics. We report five families of metrics, chosen to (i) align with prior work's expected vocabulary and (ii) surface architectural differences invisible to host-level benchmarks:

- *Task completion:* Recall against the ground-truth vulnerability set, the network-scale analogue of the sub-task / task completion rate reported by PentestGPT, VulnBot, AutoPentester, Pentest-R1, and xOffense.
- *Quality:* Precision, F1, and weighted Score (CVSS-severity-weighted Recall, §4.5). Score plays the role of AutoPentester's "vulnerability coverage" but with severity weights.
- *Multi-Hop Reach:* MHR_k for $k \in \{1, 2, 3\}$, the fraction of ground-truth vulnerabilities at `hop_depth` $\geq k$ that are reached. Existing LLM-pentest benchmarks generally report target, sub-task, or flag completion rather than recall as a function of network depth.
- *Evidence depth:* L_1 (detection), L_2 (exploitation), L_3 (exfiltration) coverage; absent from binary flag-capture benchmarks.
- *Cost envelope:* prompt+completion tokens and monetary cost (USD) from provider billing APIs, plus wall-clock time per scenario, matching CAI [9], AutoPentester, and Pentest-R1.

Compute and budget. All runs are bounded by per-phase turn caps and the concurrency limit in Phase 4. We report prompt/completion tokens, token-estimated USD cost, and actual out-of-pocket cost separately, because subscription-backed providers can make the latter lower than per-token estimates.

7. Evaluation

7.1. Overall performance

Across the 10 main scenarios, LANCE reaches 0.962 recall, 0.922 precision, and 0.940 F1, with an 86.1% CVSS-weighted score (Table 4). The result is not driven by a single easy topology: LANCE outperforms the strongest baseline row in every main scenario, with a mean absolute F1 gap of 0.547 over the per-scenario best baseline. The two scalability scenarios included in the latest export remain consistent with this trend: mean F1 is 0.909 and mean weighted score is 87.7% on `s_11`–`s_12`. The current export does not include the per-finding evidence-level fields required to recompute MHR_k, so we report those as a planned evaluator extraction rather than as headline numbers.

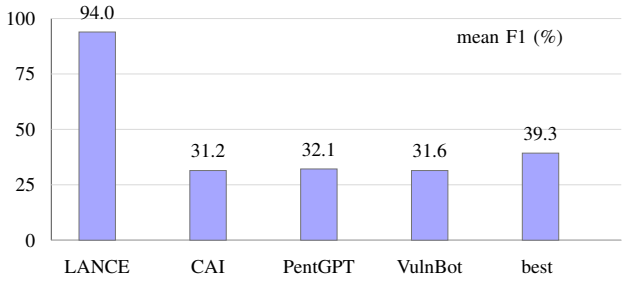

Figure 5. Mean F1 on the 10 main IoTChainBench scenarios. LANCE outperforms each adapted LLM-pentest baseline and the per-scenario best baseline row.

7.2. Cross-benchmark comparison on single-host corpora

The Vulhub run in Table 5 is useful but should be read as a lower-bound single-host sanity check, not as the main network-scale result. The distributed run processed all 191 Vulhub cases on six workers. Of these, 175 reached a completed agent run, 11 failed during environment setup, 3 failed because of provider rate limits, and 2 were skipped after the rate-limit breaker tripped. The completed runs consumed 194.5M tokens for an estimated $60.96, or roughly $0.33 per executed case.

The automatic verdict gives 69 useful findings among the 175 completed cases (39%). This undercounts success because Vulhub does not provide a uniform flag oracle: an agent can demonstrate command execution or file disclosure and still self-report failure while searching for a nonexistent flag. Re-reading the `proof.json` files identifies 15 such hidden successes, which raises the conservative lower bound to 84/175 (48%). The six strictly confirmed exploit cases are `struts2/s2-053`, `phpunit/CVE-2017-9841`, `redis/CVE-2022-0543`, `jenkins/CVE-2018-1000861`, `php/CVE-2024-2961`, and `spring/CVE-2022-22947`; the remaining useful cases are probable vulnerabilities or tool/credential-blocked findings with actionable evidence.

The AutoPenBench rows in Table 5 use the same MiniMax-M2.7 model and the same project harness, but differ in context policy. In *blind* mode, the agent receives only the target endpoint and exposed service information; it captures 7/33 flags (21.2%) and produces useful findings on 13/33 tasks (39.4%) for $4.88 and 15.5M tokens. In *informed* mode, the agent additionally receives the benchmark case id and vulnerability label; it captures 9/33 flags (27.3%) and produces useful findings on 17/33 tasks (51.5%) for $7.43 and 23.6M tokens. We therefore treat the blind row as the fair external sanity check and the informed row as an upper-bound diagnostic showing the value of vulnerability-label hints. In both modes, two real-world CVE cases fail before agent invocation due to upstream environment issues (a removed Docker base image and an unresolved restarting container).

TABLE 4. RESULTS ON IOTCHAINBENCH (10 MAIN SCENARIOS, MINIMAX-M2.7). TC = RECALL; SCORE = CVSS-WEIGHTED RECALL. BASELINE ROWS REPORT THE F1 VALUES AVAILABLE IN THE SCENARIO COMPARISON EXPORT; THE REMAINING COLUMNS ARE NOT REPORTED BY THOSE ADAPTERS.

System	TC	Prec.	F1	Score	Wins	Notes
LANCE (full)	**0.962**	**0.922**	**0.940**	**86.1%**	**10/10**	full pipeline
CAI-style adapter	–	–	0.312	–	0/10	same scenario export
PentGPT-style adapter	–	–	0.321	–	0/10	same scenario export
VulnBot-style adapter	–	–	0.316	–	0/10	same scenario export
Best baseline per scenario	–	–	0.393	–	0/10	oracle best of the three

Means over scenarios `s_1`–`s_10`*. Wins count per-scenario F1 wins over* LANCE *for baseline rows, and* LANCE *wins over the best baseline for the first row.*

TABLE 5. CROSS-BENCHMARK COMPARISON. PUBLISHED SINGLE-HOST NUMBERS ARE SHOWN FOR REFERENCE, AND LANCE (MINIMAX-M2.7) IS EVALUATED ON VULHUB IN THE LATEST RESULT BUNDLE. MODELS DIFFER ACROSS ROWS — THIS TABLE IS NOT A FAIR MODEL COMPARISON BUT CHECKS WHETHER LANCE REMAINS PLAUSIBLE ON AN ESTABLISHED SINGLE-HOST CORPUS WHILE ADDING THE NETWORK-SCALE DIMENSION. *E2E %*: FRACTION OF TASKS FULLY SOLVED. CHECKMATE USES MILESTONE-BASED PROGRESS ON A 120-CONTAINER VULHUB SAMPLE AND REPORTS ONLY A RELATIVE BENCHMARK SUCCESS-RATE GAIN OVER ITS BASELINE, NOT AN ABSOLUTE E2E RATE. "N/R" = NOT REPORTED. ⋆ = LOWER-BOUND PROOF-ORACLE ESTIMATE (SEE TEXT).

System	Model	Corpus	Sub-task %	E2E %
AutoPenBench (33 tasks)				
Llama 3.1-405B baseline [6]	Llama 3.1-405B	AutoPenBench	49.1	9.1
GPT-4o baseline [6]	GPT-4o	AutoPenBench	n/r	21.2
VulnBot [6]	Llama 3.1-405B	AutoPenBench	**69.1**	**30.3**
xOffense [11]	Qwen3-32B (LoRA)	AutoPenBench	**79.2**	n/r
LANCE (ours, blind)	MiniMax-M2.7	AutoPenBench	39.4†	21.2
LANCE (ours, informed)	MiniMax-M2.7	AutoPenBench	51.5†	27.3‡
Vulhub / Vulhub-derived single-host tasks				
Claude Code (baseline) [16]	Sonnet 4.5	Vulhub	n/r	n/r
CheckMate [16]	Sonnet 4.5 + Classical Planning+	Vulhub	n/r	**+20%**[rel]
LANCE (ours)	MiniMax-M2.7	Vulhub	n/r	**39–48%**⋆

[rel] *Relative benchmark success-rate gain over Claude Code baseline; absolute E2E rate not published by CheckMate.* † *We report useful findings as the closest sub-task analogue because our harness does not use AutoPenBench's upstream orchestrator.* ‡ *Informed mode receives the benchmark case id and vulnerability label; blind mode is the fair no-hint setting.* ⋆ *Full 191-case distributed run. The automatic useful-finding lower bound is 69/175 completed agent runs (39%); manual inspection identifies 15 additional self-reported* NO FINDING *cases with exploit evidence, raising the conservative lower bound to 84/175 (48%).*

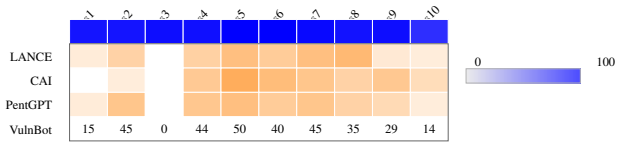

Figure 6. Per-scenario F1 heatmap on the 10 main scenarios (cell values are percentages). The hardest case for LANCE is `s_10`, a flat role-variant stress scenario, while the hub-centric `s_5`–`s_6` cases are solved perfectly.

7.3. Per-architecture analysis

The per-scenario profile in Figure 6 changes our initial expectation. The hub-centric star scenarios are not the weak point: both `s_5` and `s_6` reach 1.000 F1, suggesting that explicit graph context and deterministic probing make gateway-mediated dependencies tractable. Segmented IT/OT scenarios remain strong but not perfect (`s_4`: 0.944 F1, `s_8`: 0.933), mostly because deeper pivots increase the chance of duplicate or slightly misclassified findings.

The most difficult main scenario is `s_10`: recall drops to 0.846, precision to 0.786, F1 to 0.815, and weighted score to 71.3%. This scenario is flat rather than multi-hop, so the failure is not lateral-movement reasoning. Instead, role variants make service semantics less predictable and increase near-duplicate reports. The edge-cloud pivot (`s_7`) remains high at 0.965 F1, and the mesh smart-city case (`s_9`) reaches 0.952 F1 with perfect precision.

7.4. Evidence-level depth

The distinction still matters methodologically. The scenario-level F1 and weighted score confirm that LANCE identifies the injected vulnerabilities with high accuracy, but they do not distinguish banner-level detection from exploit proof or exfiltration. This is exactly why the benchmark stores `evidence_level` in Phase 4 deliverables: once the per-finding export is enabled, L_1–L_3 can be recomputed without rerunning the agents. Until then, we avoid using the evidence-depth argument as a quantitative claim in the results section.

TABLE 6. EVIDENCE-LEVEL COVERAGE. THE LATEST SCENARIO EXPORT REPORTS AGGREGATE TP/FP/FN SCORES BUT NOT THE PER-FINDING `EVIDENCE_LEVEL` FIELDS NEEDED TO COMPUTE L_k; THE TABLE RECORDS THE EVALUATOR STATE RATHER THAN INVENTING COVERAGE VALUES.

System	L_1 (detect)	L_2 (exploit)	L_3 (exfiltrate)
LANCE (full)	n/e	n/e	n/e
LANCE w/o Phase 5	n/e	n/e	0.00
Nmap NSE	–	–	–

n/e = not exported in the result bundle used for this draft.

TABLE 7. VULHUB FULL-RUN OUTCOME DISTRIBUTION. PERCENTAGES ARE OVER THE 175 COMPLETED AGENT RUNS UNLESS MARKED AS PRE-AGENT FAILURES.

Outcome	Cases	Share	Interpretation
Max turns	83	47%	mixed: loops plus exploited-but-no-flag cases
No finding	23	13%	includes 15 hidden proof-level successes
Blocked: credentials	23	13%	exploit path needed unavailable secret
Probable vulnerability	21	12%	actionable but not fully confirmed
Blocked: tool	19	11%	missing exploit/runtime support
Confirmed exploit	6	3%	strict automatic success
Environment failed	11	–	target did not reach the agent
Rate-limited / skipped	5	–	provider-side interruption

7.5. Per-protocol analysis

The current result bundle does not contain a protocol-keyed confusion matrix, so we do not report per-protocol recall values. The scenario pattern results nevertheless provide two useful signals. First, text-rich service mixes (HTTP/SSH/MQTT/Redis-style probes) are handled consistently: the flat, hub-centric, mesh, and edge-cloud cases all stay above 0.923 F1 except for the role-variant stress case. Second, the two segmented IT/OT scenarios remain high but below the easiest hub cases, which is consistent with the added uncertainty introduced by OT-side pivots and protocol-specific access paths.

For the final submission, the protocol breakdown should be computed directly from ground-truth categories and matched true positives, not inferred from scenario names. That export should separate at least HTTP, SSH, MQTT, database/key-value services, and OT/fieldbus-facing findings; otherwise the paper risks overstating a protocol claim that the present CSV does not support.

7.6. Failure modes and cost/time envelope

The main scenario failures are accuracy failures rather than cost failures: `s_10` dominates the error profile, with both missed findings and near-duplicate false positives caused by role variants. On Vulhub, the dominant failure mode is different. The largest bucket is MAX TURNS, and manual inspection shows that it conflates two behaviors: genuinely unproductive loops and successful exploitation followed by continued searching for a flag that the environment does not standardize. The next largest useful engineering fixes are tool coverage and credential handling: 19 completed cases were blocked by missing tools, while 23 required credentials or setup assumptions the agent did not have.

8. Discussion and Limitations

Where LLMs help, where they don't. IoTChainBench is designed to separate several failure modes that are usually collapsed into one binary success signal. The current results show strong aggregate vulnerability matching across the main scenarios, while the missing evidence/protocol export prevents us from attributing that gain to specific protocol families. The intended deployment model is therefore hybrid: LLM agents for configuration auditing and multi-step reasoning, traditional scanners for broad CVE coverage, and human operators or specialised tooling for the OT/ICS layer.

Implications for agentic systems. The harness's per-device and per-vulnerability micro-agent structure is one answer to context scaling, but it cedes cross-device inference. Designs that keep a global aggregator agent in the loop without paying the full network-context cost are an open research direction.

Limitations. (i) *Simulated infrastructure.* IoTChainBench scenarios emulate IoT devices on Linux VMs; real constrained hardware introduces embedded-specific attack surfaces (bootloaders, RTOS, firmware) we do not cover. (ii) *Evaluator leniency.* Bonus categories are accepted as non-FP even without ground-truth matches, which inflates precision. (iii) *Model variance.* The evaluator supports repeated runs, but the current result bundle reports one aggregate row per scenario and therefore does not support confidence intervals. (iv) *Prompt sensitivity.* We fix a single prompt family across models; per-model prompt tuning would likely raise scores. (v) *Static architectures.* The five patterns are fixed; real deployments evolve over time and include devices not represented here (RFID badge readers, industrial PLCs, medical devices).

9. Ethical Considerations

Isolation. All experiments run on an isolated Proxmox cluster with a dedicated bridge (`vmbr1`) and no route to external networks; the OpenWrt gateway's `WAN` interface is firewall-isolated. No live hosts, production services, or third-party infrastructure are probed.

Responsible disclosure. IoTChainBench vulnerabilities are deterministically injected into our own lab VMs via Ansible. No 0-days are introduced; all categories (misconfig, weak credentials, Terrapin CVE-2023-48795, outdated Dropbear) correspond to patterns already publicly documented. No responsible-disclosure process is triggered.

Misuse risk of released artifact. The benchmark artifact (Ansible playbooks, ground truth, evaluator) does not embed exploitation payloads for unknown vulnerabilities, and the injected weaknesses are common misconfigurations

documented in OWASP IoT Top 10. The harness uses only standard pentest tools already in widespread use. We judge the net societal benefit (reproducible evaluation of LLM pentest agents, and a concrete evaluation surface for future defenders) to exceed the incremental misuse risk.

IRB / human subjects. No human subjects are involved. No personal data is collected or processed.

LLM costs and environmental footprint. We report token-estimated cost, actual API billing, and wall-clock time with the experimental results. Energy usage is not claimed unless measured from provider- or host-side telemetry.

10. Conclusion

We presented IoTChainBench, a reproducible benchmark for evaluating LLM penetration testing agents at *network scale* on IoT infrastructure, and LANCE, a network-native multi-agent harness whose infrastructure-graph guidance and dedicated lateral movement phase keep LLM context tractable while explicitly modelling reachability. On the current result bundle, LANCE achieves 0.940 mean F1 and 86.1% weighted score on the 10 main scenarios, outperforming each adapted LLM-agent baseline in every scenario. The same run bundle also shows that the remaining evaluator work is not more model output but richer exports: MHR_k, per-protocol recall, and L1–L3 evidence coverage are already defined by the benchmark and should be emitted from matched per-finding records.

Future work. (i) *Multi-model robustness:* extending the architectural comparison to recent frontier and open-weight models to confirm model-independence of the architectural advantage; (ii) *Real hardware:* extending to embedded Zigbee, LoRaWAN, and sub-GHz attack surfaces via SDR tooling; (iii) *Defender's side:* using IoTChainBench as a benchmark for LLM-assisted IoT defense (detection, patching recommendation).

All code, scenarios, ground truth, and run logs are released at `ANONYMIZED_URL` under an open license.

References

[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.

[2] A. Happe *et al.*, "HackingBuddyGPT: An LLM-driven pentest agent," 2024. [Online]. Available: https://github.com/ipa-lab/hackingBuddyGPT

[3] G. Deng *et al.*, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, 2024.

[4] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," *arXiv:2403.01038*, 2024.

[5] O. V. Aguinaga, "Nebula: An LLM-powered penetration testing assistant," *arXiv:2412.00006*, 2024.

[6] H. He *et al.*, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," *arXiv:2501.13411*, 2025.

[7] Y. Ginige *et al.*, "AutoPentester: An LLM agent-based framework for automated pentesting," *arXiv:2510.05605*, 2025.

[8] X. Shen *et al.*, "PentestAgent: Incorporating LLM agents to automated penetration testing," in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025. doi: 10.1145/3708821.3733882.

[9] V. Mayoral-Vilches *et al.*, "CAI: An open, bug bounty-ready cybersecurity AI," *arXiv:2504.06017*, 2025.

[10] Anonymous, "Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning," *arXiv:2508.07382*, 2025.

[11] Anonymous, "xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi-agent systems," *arXiv:2509.13021*, 2025.

[12] T. Abramovich *et al.*, "EnIGMA: Interactive tools substantially assist LM agents in solving CTF challenges," in *Proc. ICML*, 2025.

[13] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *Proc. NDSS*, 2025.

[14] Anonymous, "Context relay for long-running penetration-testing agents," in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.

[15] Anonymous, "AWE: A memory-augmented multi-agent framework for autonomous web penetration testing," in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.

[16] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, "Automated penetration testing with LLM agents and classical planning," *arXiv:2512.11143*, 2025.

[17] Anonymous, "Towards automated penetration testing: Introducing LLM benchmark, analysis, and improvements," *arXiv:2410.17141*, 2024.

[18] M. Shao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[19] A. K. Zhang *et al.*, "Cybench: A framework for evaluating cybersecurity capabilities and risk of language models," *arXiv:2408.08926*, 2024.

[20] M. Bhatt *et al.*, "CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models," *arXiv:2404.13161*, 2024.

[21] V. Mayoral-Vilches *et al.*, "CAIBench: A meta-benchmark for evaluating cybersecurity AI agents," *arXiv:2510.24317*, 2025.

[22] Ludus Cloud, "Ludus: Cyber range automation on Proxmox." [Online]. Available: https://ludus.cloud

[23] M3, "GOAD: Game of Active Directory." [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD

[24] Vulhub contributors, "Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs." [Online]. Available: https://github.com/vulhub/vulhub

[25] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, 2017.

[26] W. Turton, "Hackers breach thousands of security cameras, exposing Tesla, jails, hospitals," *Bloomberg*, Mar. 2021.

[27] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[28] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proc. New Security Paradigms Workshop*, 1998.

[29] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002.

[30] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. USENIX Security Symp.*, 2005.

[31] OWASP Foundation, "OWASP Internet of Things Top 10," 2024. [Online]. Available: https://owasp.org/www-project-internet-of-things-top-10/

[32] MITRE Corporation, "MITRE ATT&CK for ICS," 2024. [Online]. Available: https://attack.mitre.org/matrices/ics/

[33] K. Stouffer *et al.*, "Guide to operational technology security," National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.

[34] S. Garcia, A. Parmisano, and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, 2020. [Online]. Available: https://www.stratosphereips.org/datasets-iot23

[35] A. Alsaedi, N. Moustafa, Z. Tari, A. Mahmood, and A. Anwar, "TON_IoT telemetry dataset: A new generation dataset of IoT and IIoT for data-driven intrusion detection systems," *IEEE Access*, vol. 8, pp. 165130–165150, 2020.

[36] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[37] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *Proc. USENIX Security Symp.*, 2014.

[38] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. NDSS*, 2016.

[39] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.

[40] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: https://github.com/OWASP/IoTGoat

[41] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: https://www.minimax.io/models/text/m27

LLM Usage Statement

LLMs were used for editorial assistance while preparing this manuscript, and all generated text was inspected by the authors to ensure accuracy, originality, and consistency with the underlying artifacts. LLMs are also part of the studied methodology: LANCE uses tool-calling LLM agents for network reconnaissance, vulnerability analysis, exploit verification, intrusion attempts, and report generation, as described in §5, §6, and §7.